# Reading the Logs

## Plenary

# Aims

## Each aim can be checked against these slides

- Familiarise with typical log lines, and determine what normal looks like.
- Programmatically find starting points: identify what does not look expected or normal.
- Programmatically expand the view to look more closely at the unexpected.
- Start to devise a chain of events.
- Consider what can be transferred from this session to other logs and settings.

# Before starting

## What to have to hand

- From Moodle: the two log files, `auth.log` and `daemon.log`, in a folder called `logs`.
- From Moodle: the script `analysis.py`, in the same folder as `logs`.
- A notepad and a pen, for the chain table and the notes on the rules.
- Every claim should rest on a line that can be **quoted**.


CONTEXT
READING
JOB A
JOB B
LIMITS


CONTEXT
READING
JOB A
JOB B
LIMITS

# Tarbet
# Marine Supplies

# Tarbet Marine Supplies

## A fictional example

- A chandlery in Oban that sells boat equipment to the public and to local boatyards.
- The online shop and the order records run on one Linux server, `shop01`. It holds **customer names, addresses and order histories**.
- Staff work from the office network, whose addresses begin `10.90.`, and log in over SSH.
- Last week a customer reported that her details were **for sale online**.

# Who else has access

## Two outside parties, both with passwords

- **Brodie Refrigeration** maintains the cold-store controllers. It logs in from its own office with a shared account, `brodie-svc`, which can use `sudo`.
- Tarbet's accountants, **Fraser & Lyle**, log in remotely as `flyle`.
- Two logs were recovered for 2 to 8 November: `auth.log` (logins and `sudo`) and `daemon.log` (the database, `cron`, and the nightly backup).

# Four questions to start the analysis

## Each question builds on the one before

- **What is normal on this server?** Read the lines first, with no tools. (Task 1)
- **Where are the starting points in so many lines?** Programmatically find who is failing to log in, since failures may mark attackers. (Job A)
- **What else has the attacker been doing?** Programmatically expand the view around each starting point, and order the events in time. (Job B)
- **What does the evidence point towards?** Reason from the chain of events, argue for a reading, and state what the evidence cannot show. (Task 4)


CONTEXT
READING
JOB A
JOB B
LIMITS

# Task 1:
# Read the lines

# Task 1: read the lines

## Groups, 4 minutes

- Open `logs/auth.log` in a text editor and read the first thirty lines.
- What is a normal login in this log? How many lines does it take?
- Does anything stand out?
- Do not run anything yet.

# Two jobs, one script

## Now both jobs answer questions 2 and 3

- **Job A**: for each source address, count the failed and the accepted logins. Output: a table, and `source_summary.csv`.
- **Job B**: for a chosen name (an address or an account), copy every line that mentions it, from both logs, into a text file.
- Two settings only change: `INTERNAL_PREFIX` and `KEYS`.

```
LOG_FOLDER = "logs"          # the folder holding auth.log and daemon.log
INTERNAL_PREFIX = "10.90."   # Job A: addresses starting like this
                             # are inside the office
KEYS = []                    # Job B: the addresses or accounts to follow, in quo
                             # separated by commas, for example ["10.1.2.3", "ali
```

# Job A (1)

## Count the failed and accepted logins

```
# ---------------- Job A: count logins by source address ----------------
failed, accepted = {}, {}
for line in open(LOG_FOLDER + "/auth.log"):
    words = line.split()
    if "Failed password" in line:
        ip = words[words.index("from") + 1]
        failed[ip] = failed.get(ip, 0) + 1
    elif "Accepted password" in line:
        ip = words[words.index("from") + 1]
        accepted[ip] = accepted.get(ip, 0) + 1
```

# Job A (2)

## Build the table and write it out

```
everyone = set(failed) | set(accepted)
table = sorted(everyone, key=lambda ip: failed.get(ip, 0), reverse=True)

out = open("source_summary.csv", "w")
out.write("source,where,failed,accepted\n")
for ip in table:
    where = "internal" if ip.startswith(INTERNAL_PREFIX) else "external"
    row = f"{ip},{where},{failed.get(ip, 0)},{accepted.get(ip, 0)}"
    out.write(row + "\n")
    print(row)
out.close()
```

# One line, worked through

## Job A on a single line

```
Nov 2 08:38:45 shop01 sshd[1128]: Failed password for cdunn from
  10.90.1.13 port 42180 ssh2
```

- **Match**: the line contains `Failed password`, so it is counted.
- **Split**: `line.split()` gives 14 words. Word 0 is `Nov`, word 8 is `cdunn`.
- **Find**: `words.index("from")` is 9, so `words[9 + 1]` is the address `10.90.1.13`.
- **Count**: `failed["10.90.1.13"]` goes up by one.

# How Job A works

## Four small ideas

- **Loop** over the file, one line at a time.
- **Match**: `if "Failed password" in line` skips every other line.
- **Split and find**: `line.split()` cuts the line into words. The address is the word after `from`, wherever `from` falls. (Some lines have `invalid user` before it.)
- **Count in a dictionary**: `failed.get(ip, 0) + 1` adds one for that address.
- `everyone` joins both dictionaries, so an address with only successes still appears in the table.

# Job B

## Find every line that mentions a name

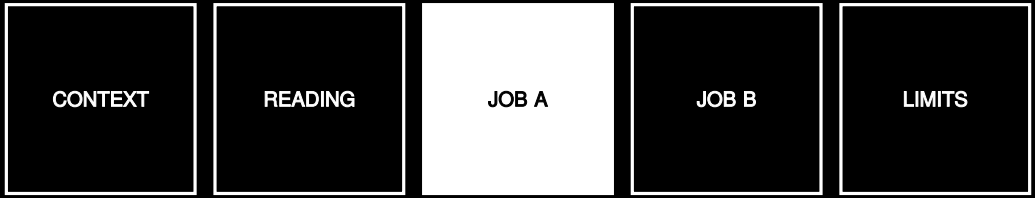
```
# ---------------- Job B: find every line that mentions a key ----------------
for key in KEYS:
    out = open("evidence_" + key + ".txt", "w")
    count = 0
    for name in ["auth.log", "daemon.log"]:
        for line in open(LOG_FOLDER + "/" + name):
            if key in line:
                out.write(line)
                count += 1
    out.close()
    print("Job B:", count, "lines mentioning", key, "written to evidence_" + key
```

- The same loop and match, but the lines are **kept**. The `auth.log` lines come first, then `daemon.log`, so the lines must be **sorted by time**.

# If it does not run

## Three usual causes

- **Wrong folder**: the terminal must be in the folder that holds `analysis.py` and `logs`.
- **Wrong command**: try `python` if `python3` is not found.
- **Broken indentation** after pasting. Python cares about the spaces at the start of a line. Use the file, not a copy from a slide.


CONTEXT
READING
JOB A
JOB B
LIMITS

# Task 2:
# Job A

# Setting INTERNAL_PREFIX

## The rule, from its own table

- The practice logs give their prefix in the scenario. Other logs do not, so the prefix has to be found.
- Look at **accepted logins**. Staff addresses log in successfully many times each. Outside sources, and the attacker, look different.
- Choose the prefix that covers the staff addresses and no outside address.
- Write the rule as a sentence: “Internal means starting . . . , because . . . ”
- Check the table: every staff row should read `internal`, and no outside row should.

# Setting KEYS

## The attacker's address and the compromised account

- **Attacker's address:** many `Failed password` lines and very few successes, outside the internal prefix. Check the whole table, not only the top row.
- **Compromised account:** open that address's evidence file. Which account has many failures, then an `Accepted password` line from the same address?
- Both go in `KEYS`, in quotes, separated by commas. Each needs a reason.

# Task 2: run Job A

## Groups, 6 minutes

- Run `python3 analysis.py`. Read the table it prints.
- Which address stands out, and **what is the evidence**?
- Three addresses are labelled `external`. Which of them would be called hostile? What else would need to be known?
- One address has no failures. Why is it in the table?


CONTEXT
READING
JOB A
JOB B
LIMITS

# Task 3:
# Job B

# Three things to check in Job B

## Before trusting an evidence file

- **A short key matches longer ones.** `"10.21.1.1"` also matches `10.21.1.10` to `10.21.1.19`. Open the file and check that it holds only the intended lines.
- **The lines are not in time order.** `auth.log` lines come first, then `daemon.log`. Sort by the time, not the text: in plain text, `Apr` sorts before `Feb`.
- **Job A counts only two kinds of line:** `Failed password` and `Accepted password`. Invalid-user, `sudo` and `cron` lines are not in the source summary. Job B finds them.

# Sorting by time

## A short addition

```
from datetime import datetime

def when(line):
    # syslog has no year: supply it (the logs are from 2026)
    return datetime.strptime("2026 " + line[:15], "%Y %b %d %H:%M:%S")

lines = open("evidence_203.0.113.17.txt").readlines()
for line in sorted(lines, key=when):
    print(line, end="")
```

# Task 3: follow the address and account

**Groups, 12 minutes**

- Set `KEYS` to the chosen address. Run the script and open `evidence_...txt`.
- Find the line where it **got in**. Which account? Add that account to `KEYS` and run again.
- Using both files, build the **chain table** on the notepad: each stage, when it happened, and one line of evidence.
- Put the lines in **time order**. What happened after the login? What ran afterwards?


CONTEXT
READING
JOB A
JOB B
LIMITS

# Task 4:
# What the evidence cannot show

# Task 4: the limits

**Groups, 5 minutes**

- What can the **table** not show?
- Brodie's account is used by both the attacker and Brodie. How can the two be told apart?
- An attacker logs in at the **first** try with a stolen password. What would Job A show for that address?
- Write **one sentence** stating the rule for "internal" and why.

# Take-aways

## Before turning to other logs

- Decide what is **normal** first, then count.
- The script **narrows the search**. What matters is decided by the analyst, and **quote the lines** that show it.
- **External is not hostile.** A count is not proof.
- Follow the **address** and the **account**. Each shows lines the other misses.
- The method transfers to other logs. The transfer section at the end shows how.

# Transfer
# to the assessment

# What the assessment asks for

## Where each output comes from

- **Source summary**: Job A's table and `source_summary.csv`, with each address marked internal or external.
- **Evidence lines**: the `evidence_...txt` files from Job B, for the attacker's address and the compromised account.
- **Table 1, incident facts**: the attacker's address, the compromised account, and the time and log line for each of: first access attempt, first successful login, privilege escalation, persistence, and the action against the customer data.
- **Table 2, attack chain**: every stage in time order, from both logs, each with a quoted line.
- **Group settings**: `INTERNAL_PREFIX` and `KEYS`, set by the group, with the reasons in a note at the top of `analysis.py`.

# Table 1: where each fact lives

## Look in the line type, then in the file

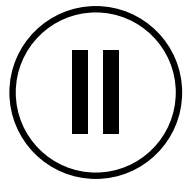
- **First access attempt**: the earliest `Invalid user` or `Failed password` line from the attacker's address, in `auth.log`.
- **First successful login**: the first `Accepted password` line from that address.
- **Privilege escalation**: a `sudo` line that runs a command as root, after that login.
- **Persistence**: a `sudo` line that changes the `crontab`, in `auth.log`, and the matching `crontab` line in `daemon.log`.
- **Action against the customer data**: a `daemon.log` job that reads or moves the database. State which moment is used: the start, or the completion.

# Transfer to other logs

## The method stays the same, the values change

- Other logs use a **different internal range**, different addresses and different accounts.
- Set `INTERNAL_PREFIX` and `KEYS` from that log set's own table, with a reason for each.
- Keep the order: learn what is normal, find starting points, expand the view, then devise the chain.

Any questions?



# Reading the Logs

## Plenary